

Test Design Document

Clinic Booking System

Student Biruk Mulatu
Student ID [STUDENT ID — fill in before submission]
Instructor Abel Tadesse
Date 4 September 2026 (revised 6 September 2026)

Sections 1–4 derive test cases for the four course rules (1, 2, 3, 3b) using the four formal techniques the assignment requires: equivalence partitioning, boundary value analysis, a decision table, and state transition testing. These four rules are the primary vehicle for the techniques — each is shaped to feed exactly one — so the working is shown in full here. Section 5 applies the same four techniques to the seven extension rules (F–L) more briefly, pointing to the derived cases in code. Section 6 adds a fifth technique, pairwise (all-pairs) testing, applied to the booking outcome. Every case listed is implemented, by the same name, under `backend/src/test/java/et/aauc/clinic/` — the map from design to code is exact, not illustrative.

1. Equivalence partitioning and boundary value analysis — Rule 1 (consultation fee by age)

Implemented in `core.FeeCalculator`. The input is a single integer, `age`.

1.1 Partitions

#	Partition	Range	Expected outcome
P1	Invalid — below range	$\text{age} < 0$	Rejected: <code>IllegalArgumentException</code>
P2	CHILD	$0 \leq \text{age} \leq 17$	Category CHILD, fee 100 ETB
P3	ADULT	$18 \leq \text{age} \leq 64$	Category ADULT, fee 250 ETB
P4	SENIOR	$65 \leq \text{age} \leq 120$	Category SENIOR, fee 150 ETB
P5	Invalid — above range	$\text{age} > 120$	Rejected: <code>IllegalArgumentException</code>

1.2 Boundaries

Three valid partitions sitting side by side produce six interior boundaries (the low and high edge of each), plus the two outer edges where valid meets invalid — eight boundary values in total, exactly the set CLAUDE.md specifies: **–1, 0, 17, 18, 64, 65, 120, 121**.

1.3 Derived test cases

Case	Input (age)	Partition / boundary	Expected	Test method
TC-F01	–1	P1, boundary just below P2	throws	<code>fee_belowLowerBoundary_ageMinus1_throws</code>
TC-F02	–50	P1 interior, far from boundary	throws	<code>fee_wellBelowRange_ageMinus50_throws</code>
TC-F03	0	P2 lower boundary	CHILD, 100	<code>fee_atLowerBoundaryOfChildBand_age0_is100</code>
TC-F04	9	P2 interior	CHILD, 100	<code>fee_withinChildBand_age9_is100</code>
TC-F05	17	P2 upper boundary	CHILD, 100	<code>fee_atUpperBoundaryOfChildBand_age17_is100</code>
TC-F06	18	P3 lower boundary	ADULT, 250	<code>fee_atLowerBoundaryOfAdultBand_age18_is250</code>
TC-F07	40	P3 interior	ADULT, 250	<code>fee_withinAdultBand_age40_is250</code>
TC-F08	64	P3 upper boundary	ADULT, 250	<code>fee_atUpperBoundaryOfAdultBand_age64_is250</code>
TC-F09	65	P4 lower boundary	SENIOR, 150	<code>fee_atLowerBoundaryOfSeniorBand_age65_is150</code>
TC-F10	90	P4 interior	SENIOR, 150	<code>fee_withinSeniorBand_age90_is150</code>
TC-F11	120	P4 upper boundary	SENIOR, 150	<code>fee_atUpperBoundaryOfSeniorBand_age120_is150</code>
TC-F12	121	P5, boundary just above P4	throws	<code>fee_aboveUpperBoundary_age121_throws</code>
TC-F13	200	P5 interior, far from boundary	throws	<code>fee_wellAboveRange_age200_throws</code>

13 cases: 3 valid partitions × 3 (low boundary, interior representative, high boundary) = 9, plus 2 invalid partitions × 2 (edge value, far-out interior value) = 4.

2. Decision table — Rule 2 (booking eligibility)

Implemented in `core.BookingPolicy`. Three binary conditions, evaluated in strict priority order $C1 \rightarrow C2 \rightarrow C3$:

- **C1** — the requested slot is still free
- **C2** — the patient has no outstanding balance
- **C3** — the request is made at least 2 hours before the slot start time

2.1 Full table ($2^3 = 8$ rules)

Rule	C1 free	C2 no balance	C3 notice ok	Action
R1	F	—	—	Reject: SLOT_UNAVAILABLE
R2	T	F	—	Reject: OUTSTANDING_BALANCE
R3	T	T	F	Reject: INSUFFICIENT_NOTICE
R4	T	T	T	Approve — appointment created in REQUESTED

Because each condition is checked in priority order with an early return, the naive $2^3 = 8$ -row truth table collapses to 4 rules with "don't care" (—) columns for conditions a higher-priority rule already decided — the same shape the implementation itself has (if (lc1) ...; if (!lc2) ...; if (!lc3) ...; approve).

2.2 Derived test cases

For every rule with a don't-care column, two tests are written — one with the don't-care condition true and one false — to prove the outcome genuinely does not depend on it (i.e. that the priority ordering really holds and a lower-priority condition can't override a higher one). Boundary value analysis is then applied separately to the one numeric condition, C3's 2-hour threshold.

Case	C1	C2	C3 / notice given	Expected	Test method
TC-B01	F	T	3h (T)	SLOT_UNAVAILABLE	<code>decision_slotUnavailable_withGoodBalanceAndNotice_rejectsWithSlotUnavailable</code>
TC-B02	F	F	30min (F)	SLOT_UNAVAILABLE (proves C1 outranks C2 and C3)	<code>decision_slotUnavailable_withBadBalanceAndInsufficientNotice_stillRejectsWithSlotUnavailable</code>
TC-B03	T	F	3h (T)	OUTSTANDING_BALANCE	<code>decision_outstandingBalance_withSufficientNotice_rejectsWithOutstandingBalance</code>
TC-B04	T	F	30min (F)	OUTSTANDING_BALANCE (proves C2 outranks C3)	<code>decision_outstandingBalance_withInsufficientNotice_stillRejectsWithOutstandingBalance</code>
TC-B05	T	T	30min (F)	INSUFFICIENT_NOTICE	<code>decision_insufficientNotice_rejectsWithInsufficientNotice</code>
TC-B06	T	T	3h (T)	Approved	<code>decision_allConditionsTrue_approves</code>

2.3 BVA on the C3 threshold (2-hour notice)

Case	Notice given	Boundary	Expected	Test method
TC-B07	exactly 2h00m	lower boundary of "sufficient"	Approved ($\geq$ is inclusive)	<code>notice_exactlyTwoHoursBeforeSlot_isSufficient_approves</code>
TC-B08	1h59m (2h – 1min)	just below the boundary	INSUFFICIENT_NOTICE	<code>notice_oneMinuteLessThanTwoHours_isInsufficient_rejects</code>
TC-B09	10h	well above the boundary	Approved	<code>notice_wellOverTwoHours_isSufficient_approves</code>

3. State transition testing — Rule 3 (appointment lifecycle)

Implemented in `core.AppointmentStateMachine`. The original course design is 5 states $\times$ 4 events. The extensions grew the machine *by extension, never by rewrite* — the original REQUESTED/CONFIRMED blocks are unchanged:

- Phase C added the `WAITLISTED` state and the `PROMOTE` event.
- Rule K added `RESCHEDULE` — a slot move that keeps the appointment in its current state, so REQUESTED and CONFIRMED each get a self-loop.
- Rule L added the terminal `OFFER_EXPIRED` state and the `EXPIRE_OFFER` event (a promoted waitlist offer not accepted within 2 hours).

Final machine: states REQUESTED, CONFIRMED, ATTENDED, CANCELLED, NO_SHOW, WAITLISTED, OFFER_EXPIRED; events CONFIRM, CANCEL, ATTEND, MARK_NO_SHOW, PROMOTE, RESCHEDULE, EXPIRE_OFFER . $7 \times 7 = 49$ pairs: 10 valid, 39 invalid .

3.1 State transition table

From \ Event	CONFIRM	CANCEL	ATTEND	MARK_NO_SHOW	PROMOTE	RESCHEDULE	EXPIRE_OFFER
REQUESTED	→ CONFIRMED	→ CANCELLED	inv	inv	inv	→ REQUESTED	→ OFFER_EXPIRED
CONFIRMED	inv	→ CANCELLED	→ ATTENDED	→ NO_SHOW	inv	→ CONFIRMED	inv
WAITLISTED	inv	→ CANCELLED	inv	inv	→ REQUESTED	inv	inv
ATTENDED (terminal)	inv	inv	inv	inv	inv	inv	inv
CANCELLED (terminal)	inv	inv	inv	inv	inv	inv	inv
NO_SHOW (terminal)	inv	inv	inv	inv	inv	inv	inv
OFFER_EXPIRED (terminal)	inv	inv	inv	inv	inv	inv	inv

Valid by row: REQUESTED 4, CONFIRMED 4, WAITLISTED 2, four terminal states 0 = 10 valid. Invalid = 49 – 10 = 39. REQUESTED and WAITLISTED are entered by construction (a fresh booking, or one made when the slot was taken), not by transition.

3.2 Derived test cases — valid transitions

Case	From	Event	Expected	Test method
TC-S01	REQUESTED	CONFIRM	CONFIRMED	transition_requestedConfirm_movesToConfirmed
TC-S02	REQUESTED	CANCEL	CANCELLED	transition_requestedCancel_movesToCancelled
TC-S03	CONFIRMED	ATTEND	ATTENDED	transition_confirmedAttend_movesToAttended
TC-S04	CONFIRMED	CANCEL	CANCELLED	transition_confirmedCancel_movesToCancelled
TC-S05	CONFIRMED	MARK_NO_SHOW	NO_SHOW	transition_confirmedMarkNoShow_movesToNoShow
TC-S09	WAITLISTED	PROMOTE	REQUESTED	transition_waitlistedPromote_movesToRequested
TC-S10	WAITLISTED	CANCEL	CANCELLED	transition_waitlistedCancel_movesToCancelled
TC-S11	REQUESTED	RESCHEDULE	REQUESTED (self-loop)	transition_requestedReschedule_staysRequested
TC-S12	CONFIRMED	RESCHEDULE	CONFIRMED (self-loop)	transition_confirmedReschedule_staysConfirmed
TC-S13	REQUESTED	EXPIRE_OFFER	OFFER_EXPIRED	transition_requestedExpireOffer_movesToOfferExpired

(TC-S06–S08 are the Rule 3b boundary cases, §3.4.)

3.3 Derived test cases — invalid transitions

Rather than hand-listing 39 rows (error-prone — easy to miss or duplicate a pair), the test generates all 49 state/event pairs and subtracts the 10 valid ones above, then runs a single parameterised test (transition_invalidPair_throwsIllegalStateException) once per remaining pair, asserting each throws IllegalStateException . A companion test (invalidStateEventPairs_containsExactlyThirtyNinePairs) asserts the generated set has exactly 39 members, so the count itself — not just each individual case — is under test. The test file also carries the hand-derivation of the "10 valid / 39 invalid" split, row by row, so the grid can be re-checked by inspection. This guarantees full coverage of the invalid region without the risk of a manually transcribed list silently missing a cell.

3.4 BVA — Rule 3b, late cancellation fee (24-hour guard)

A cancellation of a CONFIRMED appointment charges 50% of the consultation fee if made less than 24 hours before the slot start, and nothing otherwise. This is a second, independent boundary layered on the CANCEL transition.

Case	Time before slot	Boundary	Expected fee (on 250 ETB)	Test method
TC-S06	23h59m	just inside the late window	125.0 (50%)	lateCancellationFee_atTwentyThreeHours59Minutes_chargesHalfFee
TC-S07	exactly 24h00m	the boundary itself	0 (free — 24h is not "less than")	lateCancellationFee_atExactlyTwentyFourHours_isFree

TC-S08	24h01m	just outside the late window	0 (free)	lateCancellationFee_atTwentyFourHoursOneMinute_isFree
--------	--------	------------------------------	----------	---

4. Summary — course rules

Technique	Applied to	Test cases derived
Equivalence partitioning + BVA	Rule 1 (fee by age)	13
Decision table + BVA	Rule 2 (booking eligibility)	9
State transition testing	Rule 3 (lifecycle) — 10 valid + 39 invalid	10 explicit valid + 39 via one parameterised test + 1 count-check
BVA	Rule 3b (late cancellation fee)	3

5. The same four techniques applied to the extension rules (F–L)

Each extension rule is a pure function in `et.aau.clinic.core` with a class-level comment naming its technique and its boundaries, and a dedicated `*Test` class whose methods carry TC- traceability comments. The derivations are summarised here; the full case lists are in the test classes named below.

Rule	Class	Technique(s)	Partitions / rules / boundaries	Test class (cases)
F — patient registration	PatientRegistrationPolicy	Decision table (5 conditions, priority order) + BVA	C1 name present · C2 username present · C3 not taken · C4 password ≥ 4 chars · C5 DOB in the past and age ≤ 120. Boundaries: password length 3 vs 4; DOB today vs tomorrow; exactly 120y vs 120y + 1d; null password; null DOB.	PatientRegistrationPolicyTest (TC-R01–R16 + TC-R10b)
G — doctor daily load	DoctorLoad	Equivalence partitioning + BVA	Partitions AVAILABLE / NEARLY_FULL / FULL over "scheduled ÷ limit". Boundaries around the limit: one below, exactly at, one over.	DoctorLoadTest (TC-G01–G08)
H — 24-hour reminder	ReminderPolicy	Decision table (4 conditions, priority order) + BVA	C1 CONFIRMED · C2 not already reminded · C3 slot not started · C4 within 24h. Window is inclusive; boundaries 24h01m (skip), exactly 24h00m (send), 0h01m (send), slot already started.	ReminderPolicyTest (TC-H01–H12; 17 executions with the parameterised non-CONFIRMED status case)
I — insurance coverage	CoverageCalculator	Equivalence partitioning + BVA	Partitions invalid-low (< 0) / valid [0, 100] / invalid-high (> 100). Boundaries –1, 0, 1, 99, 100, 101, plus a rounding case.	CoverageCalculatorTest (TC-I01–I09)
J — no-show suspension	SuspensionPolicy	BVA on two independent axes + EP	Count threshold: 2 clear, 3 suspended, 4 suspended. 90-day window edge: 89d inside, exactly 90d inside (inclusive), 90d + 1 aged out. EP: each no-show timestamp in-window vs aged out.	SuspensionPolicyTest (TC-J01–J08)
K — reschedule	ReschedulePolicy	Decision table (3 conditions, priority order) + BVA + state transition (see §3)	C1 state reschedulable · C2 new slot free · C3 ≥ 2h notice on the new slot. BVA on the 2-hour notice: 1h59m rejects, exactly 2h00m allowed. Plus the RESCHEDULE self-loops in the state machine (TC-S11, S12).	ReschedulePolicyTest (TC-K01–K07)
L — waitlist offer expiry	WaitlistOfferPolicy	BVA on a single guard + state transition (see §3)	2-hour acceptance window: 1h59m still live, exactly 2h00m expired (inclusive), 2h01m expired. Plus the EXPIRE_OFFER → OFFER_EXPIRED transition (TC-S13).	WaitlistOfferPolicyTest (TC-L01–L04)

6. Pairwise (all-pairs) testing — self-service booking

A fifth technique, combinatorial testing, is applied to the booking outcome, which depends on six independent factors: age band (3 values), slot free (2), balance (2), notice (2), suspension (2) and coverage (3) — a $3 \times 2 \times 2 \times 2 \times 2 \times 3 = 144$ -combination space.

A greedy all-pairs generator (`docs/pairwise-booking-gen.py` , reproducible) produces a **10-row covering array** in which every pair of factor values co-occurs in at least one row — the standard all-pairs criterion — seeded with the three approve-path rows so each age × coverage fee combination is also checked on a successful booking. One supplementary row is added so `OUTSTANDING_BALANCE` appears as an actually-reported reason (in the covering array it is always masked by a higher-priority condition). Total: **11 cases**, TC-P01–P11, in `BookingPairwiseIT` . The factor table, the covering array, the expected-outcome derivation and the observation that "pairwise coverage of the inputs does not imply coverage of the outcomes" are written up in `docs/pairwise-booking.md` .

7. Where each technique is demonstrated (Part B checklist)

Technique	Primary demonstration	Further instances
Equivalence partitioning	Rule 1 (§1)	Rules G, I; each no-show timestamp in Rule J
Boundary value analysis	Rule 1 (§1), Rule 3b (§3.4)	Rule 2 C3 (§2.3); Rules F, G, H, I, J, K, L
Decision table	Rule 2 (§2)	Rules F, H, K
State transition testing	Rule 3 (§3) — 7 × 7 grid	Rules K and L extend the same machine
Pairwise / combinatorial	Booking outcome (§6)	—